Training Transformers with Enforced Lipschitz Constants

Laker Newhouse [*] MIT CSAIL
R. Preston Hess [*] MIT BCS
Franz Cesista [*] Ateneo de Manila University
Andrii Zahorodnii MIT BCS
Jeremy Bernstein MIT CSAIL
Phillip Isola MIT CSAIL

Abstract

Neural networks are often highly sensitive to input and weight perturbations. This sensitivity has been linked to pathologies such as vulnerability to adversarial examples, divergent training, and overfitting. To combat these problems, past research has looked at building neural networks entirely from Lipschitz components. However, these techniques have not matured to the point where researchers have trained a modern architecture such as a transformer with a Lipschitz certificate enforced beyond initialization. To explore this gap, we begin by developing and benchmarking novel, computationally-efficient tools for maintaining norm-constrained weight matrices. Applying these tools, we are able to train transformer models with Lipschitz bounds enforced throughout training. We find that optimizer dynamics matter: switching from AdamW to Muon improves standard methods—weight decay and spectral normalization—allowing models to reach equal performance with a lower Lipschitz bound. Inspired by Muon's update having a fixed spectral norm, we co-design a weight constraint method that improves the Lipschitz vs. performance tradeoff on MLPs and 2M parameter transformers. Our 4-Lipschitz transformer on Shakespeare text reaches validation accuracy 60%. Scaling to 145M parameters, our 600-Lipschitz transformer reaches 21% accuracy on internet text. However, to match the NanoGPT baseline validation accuracy of 39.4%, our Lipschitz upper bound increases to 10^{274}. Nonetheless, our Lipschitz transformers train without stability measures such as layer norm, QK norm, and logit tanh softcapping.

1 Introduction

Lipschitz constants for neural networks—intuitively, bounds on the sensitivity of each component and of the overall model to input perturbations—are of interest for their effect on generalization and robustness [Bartlett et al., 2017, Tsuzuku et al., 2018] and for applications such as differential privacy [Béthune et al., 2024]. Seminal work [Arjovsky et al., 2016, Cisse et al., 2017, Yoshida and Miyato, 2017, Anil et al., 2019] enforces Lipschitz constants beyond initialization for MLPs, RNNs, and GANs, but for transformers, the closest related work, LipsFormer [Qi et al., 2023], does not constrain the weight matrices during training. Without constraints, large-scale transformer training may encounter instabilities, which has been attributed to attention and output logits growing too large [Wortsman et al., 2024, Dehghani et al., 2023]. Can enforced Lipschitz constants benefit transformers, too? Specifically, we ask:

Can transformers with small, enforced Lipschitz bounds perform well?
How does the weight constraint method affect the Lipschitz-to-performance tradeoff?

[*]Equal contribution. Correspondence to `lakern@mit.edu`

Preprint. Under review.

Enforcing Lipschitz bounds on a transformer is challenging because transformers include components that are not globally Lipschitz, such as self-attention [Kim et al., 2021]. We build on Large et al. [2024] who, similar to LipsFormer, enable Lipschitz continuity by reparameterizing residual connections and modifying self-attention; however, the full story is elusive. LipsFormer goes further than Large et al. [2024] to eliminate layer norm [Ba et al., 2016], but the official implementation may make Lipschitz bounds impossible by setting $\epsilon = 0$ in QK norm [Henry et al., 2020]. In contrast, we remove activation normalization to explore whether training can proceed with no stability measures.

To develop a toolkit for training transformers with an enforced Lipschitz constant, in Section 3 we compare several methods for constraining weight norm. Surprisingly, we find that optimizer choice matters: standard methods such as weight decay [Krogh and Hertz, 1991] and spectral normalization [Yoshida and Miyato, 2017] improve a Lipschitz vs. performance tradeoff more with Muon [Jordan et al., 2024b] than with AdamW [Loshchilov and Hutter, 2019]. We see improvement under Muon for MLPs trained on CIFAR-10 [Krizhevsky, 2009] and corroborate it on 2M parameter transformers trained on Shakespeare text [Karpathy, 2022].

Beyond standard methods, we are inspired by a property of Muon—its weight updates have small, known spectral norm—to design a weight constraint method called *spectral soft cap*, which enforces a desired maximum spectral norm σ_{max} by approximating the map $\sigma \mapsto \min(\sigma_{\text{max}}, \sigma)$ on all singular values σ in parallel by iterating odd polynomials on the weights. Theorem 3.1 proves that spectrally capping the singular values bounds weight norm when training with Muon; we provide no theoretical guarantee for AdamW because the spectral norm of its update is not controlled. For AdamW, we explore a second technique that may be better suited to low stable rank updates, although we do not provide provable guarantees. At every step, this technique finds the largest weight singular value and sets it to σ_{max}. In analogy with a hammer that strikes the nail that sticks out the most, we call this technique *spectral hammer*. Our experiments suggest that the most effective combination is Muon with spectral normalization or spectral soft cap, while from Adam the only technique that elicits a competitive performance vs. Lipschitz tradeoff is spectral hammer.

In Section 4, we scale up enforced weight constraint methods to the NanoGPT speedrun benchmark [Jordan et al., 2024a], training 145M parameter transformers to competitive performance without layer norm or QK norm. We train a 600-Lipschitz transformer to 21.2% validation accuracy, compared to the non-Lipschitz baseline of 39.4% validation accuracy. However, to reach a competitive accuracy of 38.2%, our global Lipschitz upper bound becomes astronomical at 10^{127}. While Fazlyab et al. [2019] describe ways to tighten Lipschitz bounds, inspecting the maximum activation norms reveals that our model operates far from the worst case. On a particular batch of 393K tokens, the non-Lipschitz baseline has maximum activation entry 148,480 while the 10^{127}-Lipschitz transformer has maximum activation entry 96.5. Empirically small activations in Lipschitz-constrained transformers may present an opportunity for low-precision training and inference.

Our contributions are as follows:

- We train transformers with enforced Lipschitz constraints up to 145M parameters, including a 600-Lipschitz transformer that achieves 21% accuracy on FineWeb10B internet text and a 4-Lipschitz transformer that achieves 60% accuracy on Shakespeare text.
- We present evidence that weight decay and spectral normalization yield greater benefits when trained with the Muon optimizer compared to AdamW, matching accuracy with smaller Lipschitz bound. We verify standard robustness properties hold when training with Muon.
- We introduce two weight norm constraint techniques: *spectral soft cap* and *spectral hammer*. Out of weight regularization methods for AdamW, spectral hammer elicits the most competitive Lipschitz-constrained performance. For Muon, we prove spectral soft cap bounds weight norm and find that it performs similarly or slightly better than spectral normalization.

2 Related work

There is by now a large literature on the myriad potential and realized benefits of Lipschitz neural networks [Béthune et al., 2022, Béthune, 2024, Rosca et al., 2020]. Input–output Lipschitz certificates may be useful in deployment scenarios where there is a benefit to having strong robustness with respect to input perturbations. Examples include robotic control [O'Connell et al., 2022, Wang et al., 2020], classification in the presence of adversarial input perturbations [Szegedy et al., 2014], and in

protocols for AI safety [Brown-Cohen et al., 2024]. Input–output Lipschitz certificates are also used in certain generalization guarantees for deep networks [Bartlett et al., 2017, Neyshabur et al., 2018, Dherin et al., 2022].

Similarly, weight–output Lipschitz certificates may be useful in situations where there is an interest in perturbing the weights of a neural network without incurring unstable output behaviour. A prime example is for stable [Qi et al., 2023, Flynn, 2017] and scalable [Large et al., 2024] training. A second example is for the design of differentially private training algorithms where there is a need to add carefully calibrated noise to the network weights [Béthune, 2024, Béthune et al., 2024]. And a third example is to aid in weight quantization [Elthakeb et al., 2020, Weng et al., 2020].

In fact, there is a close theoretical link between input–output Lipschitzness and weight–output Lipschitzness in deep learning [Large et al., 2024, Béthune, 2024]. The reason is that when we compose two subnetworks, Lipschitzness with respect to the weights of the first subnetwork depends upon the degree of input–output Lipschitzness of the second subnetwork.

Various techniques have been proposed for producing neural networks amenable to Lipschitz certification. These include techniques for modifying the network architecture and training process to improve the resulting Lipschitz properties. For example, spectral normalization [Miyato et al., 2018, Gogianu et al., 2021] has been proposed as a means to control the Lipschitz properties of individual weight matrices. New nonlinearities [Anil et al., 2019] and normalization layers [Qi et al., 2023] have also been proposed.

Furthermore, given a trained model of a given architecture, various techniques have been proposed for deriving Lipschitz certificates. Deriving the exact Lipschitz constant (i.e. the least upper bound) is known to be computationally hard [Katz et al., 2017, Virmaux and Scaman, 2018, Weng et al., 2018] so researchers settle for producing slacker upper bounds. One approach to producing upper bounds—used in this paper for simplicity—is to obtain Lipschitz statements for each component in the architecture and add or multiply them as appropriate to compose them [Szegedy et al., 2014]. However, tighter approaches have also been proposed: Weng et al. [2018, 2019] provide examples.

3 Weight norm constraints to enforce Lipschitz constraints

A function $f(x)$ has Lipschitz constant K under a norm $\|\cdot\|$ if it satisfies $\|f(x_1) - f(x_2)\| \leq K \cdot \|x_1 - x_2\|$ for all inputs x_1, x_2. For neural networks, the most common operation is matrix multiplication which has ℓ_2 Lipschitz constant equal to the spectral norm of the weight matrix. Constraining the spectral norm of weight matrices is not new, with past work primarily exploring weight decay, spectral normalization, and orthogonal constraints [Krogh and Hertz, 1991, Yoshida and Miyato, 2017, Miyato et al., 2018, Gouk et al., 2021, Jianlin, 2024]. These methods have been tested with the AdamW optimizer and have shown benefits for generalization and adversarial robustness [Bartlett et al., 2017, Tsuzuku et al., 2018]. The Muon optimizer introduces new possibilites by ensuring small, fixed-norm weight updates. Inspired by this property, we revisit existing methods and develop new methods for constraining weights. We ask the question:

What is the best way to enforce weight norm constraints throughout training?

We compare seven methods based on how well they 1) maintain high performance, 2) enforce weight norm constraints, and 3) trade off performance with a Lipschitz bound. To summarize our conclusions, we find that the Muon optimizer achieves lower Lipschitz constants and better performance compared to AdamW. Among the constraint methods, our experiments suggest that spectral soft cap, spectral hard cap, and spectral normalization meet these criteria best.

Muon enables hard weight constraints. Unlike in AdamW, the weight update norm in Muon is bounded by the learning rate—if its orthogonalizing polynomial never exceeds 1. We follow [You, 2025] to ensure this property in our experiments. Pethick et al. [2025] noted that bounded weight update spectral norm upgrades weight decay with parameter λ to enforce a *strict* spectral norm constraint of $1/\lambda$. The reason is that an equilibrium occurs between the update step and weight decay when the weight norm w satisfies $w = w(1 - \lambda\eta) + \eta$ for learning rate $\eta > 0$. See Appendix A for details. We hypothesize that this property may explain our evidence that Muon, compared to AdamW, improves the Lipschitz vs. performance tradeoff for standard methods such as weight decay.

A spectral generalization of weight decay. Weight decay can be viewed as a special case of an *odd polynomial iteration* applied to the weights. Odd polynomials are special because they act directly on the singular values: $p(U\Sigma V^\top) = Up(\Sigma)V^\top$, where $U\Sigma V^\top$ is a singular value decomposition. The odd polynomial for weight decay is $p(x) = (1 - \eta\lambda)x$, where η is the learning rate and λ is the weight decay. Cisse et al. [2017] explored an orthogonalizing polynomial $p(x) = (1 + \beta)x - \beta x^3$, but Miyato et al. [2018] note that pressuring all singular values toward one limits information in the spectrum. Their method, *spectral normalization*, enforces norm constraints while allowing singular values less than 1 [Gouk et al., 2021], but normalization accomplishes the constraint by scaling down the entire spectrum. This global effect motivates a more targeted approach : penalizing only the singular values that are too large, leaving smaller ones untouched. For a desired maximum norm $\sigma_{\text{max}} \geq 0$, an idealized penalty would apply $\min(\sigma_{\text{max}}, \sigma)$ to the singular values, but exactly computing the SVD is slow. Odd polynomial iterations serve as a fast and effective approximation. We contribute a family of such approximations called *spectral soft cap* that contains weight decay as a special case. The derivation and discussion is in Appendix A.

3.1 Methods for controlling weight norm

We are interested in controlling the $\text{RMS} \to \text{RMS}$ operator norm—a rescaled spectral norm—which has emerged as natural for deep learning [Yang et al., 2024, Bernstein and Newhouse, 2025]. Unit $\text{RMS} \to \text{RMS}$ norm is equivalent to a spectral norm of $\sqrt{d_{\text{out}}/d_{\text{in}}}$ for a weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. In what follows, we denote the principal singular vector subspace with singular value $\sigma_1 \geq 0$ by $\sigma_1 u_1 v_1^\top$, computed via power iteration. We briefly review some known methods to constrain weight norm, then introduce two new methods called spectral capping and spectral hammer.

Weight decay, or Frobenius norm regularization, maps $W \mapsto (1 - \lambda\eta)W$ where $\lambda > 0$ is the decay parameter and $\eta > 0$ is the learning rate, guaranteeing a norm bound in conjunction with Muon.

Spectral weight decay, or spectral norm regularization, targets only the top singular value, mapping $W \mapsto W - \lambda\sigma_1 u_1 v_1^\top$ where $\lambda > 0$ is the decay parameter [Yoshida and Miyato, 2017, Jianlin, 2024].

Spectral normalization, originally introduced in GAN training [Miyato et al., 2018], guarantees a spectral norm bound by mapping $W \mapsto \frac{W}{\sigma_1}$.

Stiefel manifold projection pressures all singular values toward 1 using an odd polynomial iteration $W \mapsto p(W)$, leaving open the choice of polynomial p. We follow You [2025] whose polynomial converges very rapidly rather than the polynomial from [Cisse et al., 2017]. Although Stiefel manifold projections usually refers to projections for rectangular matrices, with a slight abuse of notation we use it to describe this operation on both square and rectangular matrices.

We extend these ideas with two new methods:

Spectral hammer is similar to spectral weight decay, but sets the top singular value to σ_{max} by mapping $W \mapsto W + (\sigma_{\text{max}} - \sigma_1)u_1 v_1^\top$—so to speak, a hammer that strikes the nail that sticks out the most. Spectral hammer does not guarantee the spectral norm stays below $\sigma_{\text{max}} > 0$ because multiple singular vectors may increase per update. Spectral hammer is better suited to low stable rank weight updates as are common in Adam [Zhao et al., 2024]. Muon's update is always high stable rank.

Spectral capping is co-designed for Muon's high stable rank update, smoothly approximating the map $\sigma \mapsto \min(\sigma_{\text{max}}, \sigma)$ for all singular values in parallel. Rather than rely on costly SVDs, it uses an odd polynomial approximation. The primary variant we experiment with is called *spectral soft cap* because it applies a loose approximation $p_2(p_1(x))$, where $p_1(x) = x - \alpha x^3$ and $p_2(x) = x + \alpha x^3$ with strength parameter $\alpha \geq 0$, as depicted in Figure 4 in Appendix A. To incorporate weight decay as a special case, we may first apply $p_0(x) = (1 - \lambda\eta)x$. This composition is designed to decay a singular value very little when $\sigma \ll \sigma_{\text{max}}$, but when $\sigma = \sigma_{\text{max}}$ to decay it as strongly as necessary to counteract Muon's known update norm, strictly enforcing a weight norm bound. When the learning rate is scheduled, finding the minimal $\alpha \geq 0$ that bounds the weight norm avoids accumulating error.

Theorem 3.1 (Spectral soft cap bounds spectral norm.)**.** *Given a desired maximum spectral norm $\sigma_{max} \geq 0$, learning rate $\eta \geq 0$, weight decay $\lambda \geq 0$, and weight matrix with bounded norm $\|W\|_* \leq \sigma_{max}$, there is a minimal $\alpha \geq 0$ such that Muon's update step followed by spectral soft cap preserves $\|W\|_* \leq \sigma_{max}$. Calculating α involves solving the roots of a quartic polynomial.*

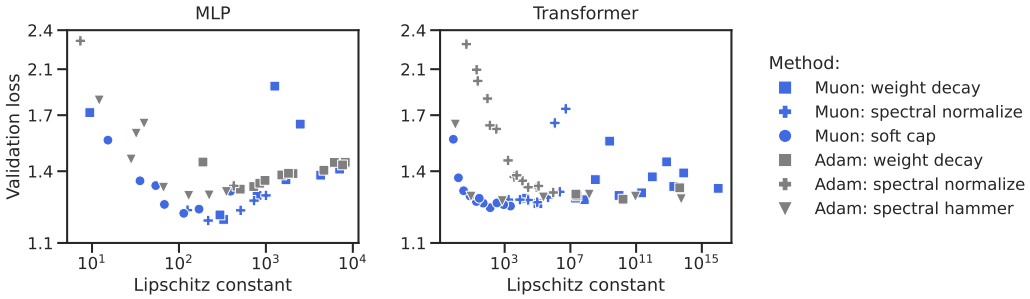

Figure 1: **Using Muon instead of AdamW improves the Lipschitz vs. performance tradeoff for standard weight regularization techniques.** We train 1600 MLPs on CIFAR-10 (left) and 800 transformers on Shakespeare text (right), varying the optimizer and weight constraint method. Weight decay and spectral normalization reach better loss with lower Lipschitz constant when using Muon. Two weight constraint methods that we design—spectral soft cap and spectral hammer—are also promising. See Appendix D for experimental details.

Appendix A gives the proof. A second variant of spectral capping is *spectral hard cap*, which uses four quintic polynomials that closely approximate $\min(\sigma_{\max}, \sigma)$, as depicted in Figure 5 in Appendix A. Because this approximation is fixed, errors can compound later in training when the learning rate is scheduled to 0.

Together, these methods cover a range of tradeoffs between strict norm enforcement, preserving the spectrum, and computational efficiency. There may be other, better approaches, and we think exploring alternatives is an exciting direction. Note that spectral soft cap and spectral hard cap are designed to be compatible with Muon and therefore were not applied to tests with AdamW.

3.2 AdamW and Muon: comparing weight constraint methods

In Figure 1, we run a sweep to map the tradeoff frontier between validation loss and Lipschitz constant across our methods. The Lipschitz constants are calculated with respect to the $\mathrm{RMS} \to \mathrm{RMS}$ operator norm (Section 3.1). For a ReLU MLP, the Lipschitz constant is the product of the $\mathrm{RMS} \to \mathrm{RMS}$ norms of its weights. For a transformer, the Lipschitz constant is calculated as described in Section 4.2. Muon consistently achieves both lower validation loss *and* lower Lipschitz constants than AdamW, a trend that holds for MLPs on CIFAR-10 and transformers on Shakespeare text. This result motivates our choice to adopt Muon for larger-scale experiments. Spectral normalization and spectral soft cap appear to make the most efficient use of a Lipschitz budget. Spectral hammer–which is designed for AdamW's low stable rank weight updates–shows competitive performance but does not enforce a Lipschitz constant, limiting its reliability for settings where constraint enforcement is critical. The sweep includes 2400 training runs, where for each method we display only the best validation loss per bin of Lipschitz constant.

3.3 Adversarial robustness of Lipschitz networks

Prior work [Cisse et al., 2017, Huang et al., 2021] suggests that a neural network's adversarial robustness is related to its Lipschitz constant, making Lipschitz control a potential path for developing models with high certified accuracy. We confirm this relationship holds for MLPs trained with Muon and spectral soft cap. Figure 2 compares the accuracy of two trained MLPs under various budgets of adversarial perturbation ϵ. The pair depicted has matching accuracy $\approx 45\%$ but different Lipschitz constants: 15.2 (Muon + spectral soft cap) vs. 7618.8 (AdamW + weight decay).

While both models achieve similar accuracy without perturbation, the Lipschitz-constrained MLP (quantified in the $\mathrm{RMS} \to \mathrm{RMS}$ operator norm, Section 3.1) trained with Muon exhibits a smoother dropoff in accuracy and confidence across the 2000 CIFAR-10 test set images as the adversarial attack increases its l_2 perturbation. Larger values of ϵ are required to fool the Lipschitz-constrained network (Figure 2, Left).

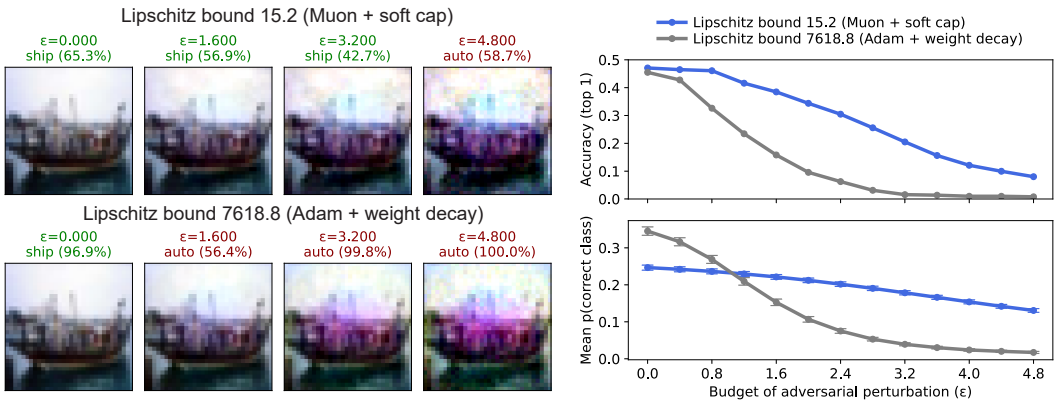

Figure 2: **Networks trained with Muon and spectral soft cap have lower Lipschitz bounds and are more adversarially robust.** Lower Lipschitz constants have been linked to greater adversarial robustness [Cisse et al., 2017, Huang et al., 2021]. To assess this effect in our models, we train a CIFAR-10 MLP with a Lipschitz constant of 15.2 (Muon + spectral soft cap), which matches the 45% clean accuracy of a baseline model (AdamW + weight decay) that has a a higher Lipschitz constant of 7618.8. Left: Example adversarial attacks with different ℓ_2 budget $\epsilon \geq 0$ for the perturbation. Top right: We quantify adversarial robustness across 2000 test images by the top-1 accuracy as a function of ϵ. The Lipschitz-constrained network trained with Muon and spectral soft cap maintains a higher accuracy for larger values of ϵ. Bottom right: the mean probability of the correct class in the Lipschitz-constrained network starts lower than that of the baseline model, but degrades slowly under increasing ϵ. By contrast, the baseline model peaks higher for $\epsilon = 0$, but drops off sharply.

3.4 Comparing weight constraint methods within Muon

In Figure 3, we use Muon alongside all seven weight constraint methods to identify which approaches best meet three goals: 1) enabling us to define a Lipschitz constant before training, 2) enforcing that constant throughout training, and 3) matching or exceeding the performance of standard weight decay. We tested each weight constraint method on a 3-layer MLP with hidden dimension 256, trained with Muon on CIFAR-10 classification. Full experimental details are in Appendix D.

In Figure 3 (left panel), we see that spectral normalization, spectral soft cap, and spectral hard cap define the frontier of the Lipschitz vs. validation loss tradeoff. In Figure 3 (middle), we select the parameter settings with the highest validation accuracy for further analysis. Spectral normalization, spectral soft cap, and spectral hard cap are the only methods to reach within 1% validation accuracy of the baseline (Muon with weight decay). Other methods reached within 4% accuracy.

Figure 3 (right panel) visualizes how the norm of the hidden weight matrix evolves during training. All methods but spectral hammer retain the weight norm at or under its target. In contrast, spectral hammer exceeds its target but begins to converge near the end, indicating that it could be a promising technique under Muon, too, on longer training runs that also schedule learning rate to 0. However, within our 50-epoch window, it fails to reliably control the Lipschitz constant. Spectral weight decay does not enforce predefined Lipschitz constants, so it is not plotted.

We select spectral soft cap, spectral hard cap, and spectral normalization for our transformer training experiments. While all weight constraint methods could potentially benefit from combining with standard weight decay, we want to isolate the effect of each one.

4 Transformers with enforced weight constraints

To develop a toolkit for training Lipschitz-constrained transformers, we begin with a discussion of how to handle residual connections and self-attention (Section 4.1). In Section 4.2, we explain how to calculate a Lipschitz bound for a transformer. In Section 4.3, we find that spectral normalization and spectral soft cap perform well on 2M parameter transformers trained on Shakespeare text. In Section 4.4, we scale up to a transformer with 145M parameters trained on FineWeb10B internet text

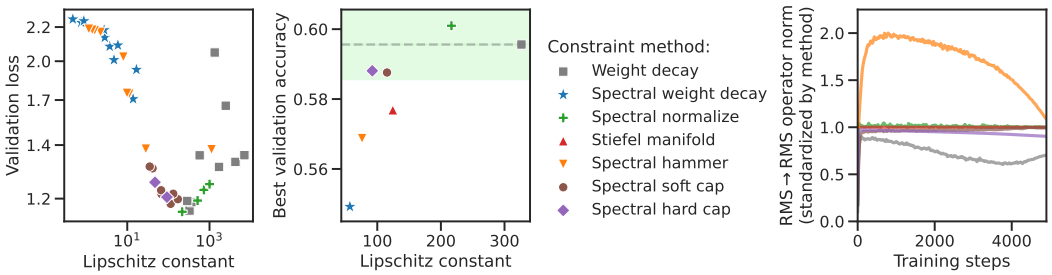

Figure 3: **Left: Weight constraint methods designed for Muon lie on the frontier of the Lipschitz vs. loss tradeoff.** Each point shows the lowest validation loss achieved at a given Lipschitz constant across all MLP runs on CIFAR-10. In the low loss regime, staying on the tradeoff frontier requires either spectral normalization or spectrally capping the singular values. **Middle: Spectral normalization and spectral capping match baseline with lower Lipschitz constant on CIFAR-10.** Each method comes within 1% accuracy (shaded green region) and has lower Lipschitz constant. **Right: RMS $\to$ RMS operator norm of hidden layers over training for the best networks.** The norm is standardized so that the weight constraints all target 1. Spectral normalization and Stiefel manifold projection strictly meet this target. Weight decay, spectral soft cap, and spectral hard cap stay below the target, while spectral hammer fails to remain constrained.

[Penedo et al., 2024]. To combat the undertuned baseline problem, we begin from the competitive benchmark of NanoGPT speedrunning [Jordan et al., 2024a].

4.1 Breaking the multiplication barrier?

A major triumph of Large et al. [2024] is to create transformers with a depth-independent Lipschitz constant. However, their approach relies on activations having unit RMS norm. Because we will later relax this constraint, our transformers will typically *not* have a depth-independent Lipschitz bound. Nonetheless, we use their two architectural suggestions.

Reparameterizing residual connections. The classic residual connection from He et al. [2016] defines the update $x + \mathsf{block}(x)$, but even a 1-Lipschitz block can exponentially increase the Lipschitz constant: $x + \mathsf{identity}(x)$ doubles at every layer. Large et al. [2024] use a convex combination

$$\frac{N-1}{N} \cdot x + \frac{1}{N} \cdot \mathsf{block}(x) \tag{1}$$

to break this multiplication barrier, where N is the number of layers. The residual connection will be 1-Lipschitz if the block is 1-Lipschitz. See Proposition 4 of Large et al. [2024]. Our experiments use this convex parameterization. However, the bound breaks down if activation norms exceed 1. We are unable to attain high performance without relaxing the 1-Lipschitz constraint. Therefore we do not fully break the multiplication barrier: deeper networks can accrue astronomical Lipschitz bounds.

Attention with $1/d$ scaling. The original multihead attention of Vaswani et al. [2017] has no global Lipschitz bound [Kim et al., 2021]. In a footnote, Vaswani et al. [2017] indicate that they chose $1/\sqrt{d}$ scaling because two random vectors u, v with mean 0 and variance 1 will have a dot product $u \cdot v$ with mean 0 and variance the dimension d. But key and query vectors may align more than at random. Perfect alignment would suggest $1/d$ scaling. Large et al. [2024] prove that $1/d$ scaling in the softmax, together with a factor of $\frac{1}{3}$, makes functional attention,

$$\frac{1}{3} \times \text{softmax}\left(\frac{QK^\top}{d}\right) V, \tag{2}$$

1-Lipschitz if the input norms are 1. The Lipschitz constant is with respect to the $\|\cdot\|_{\text{RMS}\infty}$ norm, the max RMS norm of any token. See Proposition 7 of Large et al. [2024]. Finally, while Large et al. [2024] retains layer normalization, we remove it so that every operation is Lipschitz continuous.

4.2 Calculating the Lipschitz constant of a transformer

To test whether transformers with small Lipschitz constant can perform well, we would like to know what weight norm to enforce to end up with a desired Lipschitz bound. This section sketches an

algorithm for bounding the Lipschitz constant of a transformer given its weight norms; the full algorithm is in Appendix B. Our bound tightens Theorem 2 from LipsFormer [Qi et al., 2023] by accounting for each weight norm individually, rather than working with the max weight norm across residual blocks. Fazlyab et al. [2019] suggest ways to tighten a global bound like ours, which we leave to future work. To bound self-attention, we extend Proposition 7 from the modular norm paper [Large et al., 2024] to when the input is no longer unit norm, an essential concession to reach different regions of the Lipschitz vs. performance tradeoff.

Our Lipschitz bounds are with respect to the max RMS norm over token positions, denoted $\|\cdot\|_{\infty\mathrm{RMS}}$.

Step 1: Bound activation norms. Our Lipschitz bound for rescaled dot-product attention relies on the activation norms remaining bounded. Therefore, the algorithm begins by computing a per-layer bound for the maximum $\|\cdot\|_{\infty\mathrm{RMS}}$ norm of activations. This bound comes from combining residual connections with per-block maximum activation increases, found for an MLP by multiplying its two weight norms and for attention by multiplying its W_V and W_O weight norms. Our MLPs can slightly decay activation norm because we scale GeLU down by its maximum derivative, $\mathsf{GeLU}/1.1289$.

Step 2: Bound the Lipschitz constant. Suppose the Lipschitz constant prior to reaching a particular layer is L. The Lipschitz constant after a residual connection is at most $(1-\alpha)L + \alpha \cdot L \cdot L_{\mathsf{block}}$. To compute a block's Lipschitz constant L_{block}, for MLPs we calculate $\|W_{\mathsf{in}}\|_{\mathrm{RMS}\to\mathrm{RMS}} \cdot \|W_{\mathsf{out}}\|_{\mathrm{RMS}\to\mathrm{RMS}}/1.1289$ due our rescaled GeLU, and for attention we use Theorem B.1.

4.3 Shakespeare Transformer

Before scaling to NanoGPT, we explore the Lipschitz vs. performance tradeoff for transformers at a smaller scale. To narrow our aim, we want a transformer that has small Lipschitz constant *at every layer*. Béthune et al. [2022] comments that any L-Lipschitz classifier can be made 1-Lipschitz by dividing the logits by L, but we do not downscale our final logits, although scaling temperature during training could have beneficial effects [Agarwala et al., 2023]. All experimental details are available in Appendix D.

We can train a competitive 4-Lipschitz Shakespeare transformer. Our 4-Lipschitz transformer reaches validation loss $1.29 < 1.47$ from the baseline in Karpathy [2022], although the baseline may not be tuned carefully. Our model has dimension 256, depth 3, and was trained for 2000 steps with Muon; the baseline has dimension 384, depth 6, and was trained for 5000 steps with AdamW. Our transformer uses no layer normalization. To achieve this performance requires relaxing the maximum weight norm σ_{max} to around 2. The best validation loss from our sweep was 1.20 with a 131-Lipschitz transformer. While gains may be attributed to hyperparameters or the choice of optimizer, we nonetheless end up with one construction of our aim: a performant transformer with a small, enforced Lipschitz bound.

4.4 Scaling to NanoGPT

We validate our method by training a transformer with 145M parameters on the NanoGPT speedrun benchmark [Jordan et al., 2024a], which is built on top of [Karpathy, 2022]'s reproduction of GPT-2. The baseline is competitively optimized to reach validation loss 3.28 in the shortest wallclock time. The latest record as of February 1, 2025 requires only 1770 training steps, or 3 minutes of training on an 8xH100, and achieves a validation accuracy of 39.4%. We implement our methods on top of this baseline while keeping all other training methods fixed. We report the validation loss and the validation accuracy as primary comparison metrics.

To implement our method, we first remove speedrun-specific optimizations such as skip connections and learnable scale parameters. We remove the layernorms used for pre-normalization, the logit tanh softcap, and the QK norms in the attention layers. We also replace the ReLU^2 activations with $\mathsf{GeLU}/1.1289$—making the activation function Lipschitz continuous. We reparametrize the residual connections and attention layer as in Equations (1) and (2). At initialization, we project the linear weights to be semi-orthogonal and normalize the embeddings to have RMS norm 1. Finally, we explicitly extend beyond the closest related work, LipsFormer [Qi et al., 2023], by enforcing weight norm constraints throughout training: we cap the RMS norm of embeddings to 1 and apply one of the weight constraint methods from Section 3.1 to all other weights after every training step.

Transformer Architecture	Lipschitz Bound	Number of Steps	Weight Constraint	Validation Accuracy (↑)	Validation Loss (↓)	Activation Max Entry
Baseline (speedrun)	∞	1,770	none	**0.394**	**3.280**	148,480
Baseline (Karpathy)	∞	17,000	none	-	**3.280**	-
LipsFormer	10^{130}	1,770	none	0.301	4.130	61.1
Ours ($\sigma_{\text{max}} = 1$)	600	1,770	spectral normalize	0.212	5.047	14
Ours ($\sigma_{\text{max}} = 8$)	10^{118}	1,770	spectral soft cap	**0.365**	**3.569**	54.25
Ours ($\sigma_{\text{max}} = 8$)	10^{56}	1,770	spectral hard cap	0.354	3.670	107
Ours ($\sigma_{\text{max}} = 8$)	10^{130}	1,770	spectral normalize	0.360	3.607	100
Ours ($\sigma_{\text{max}} = 16$)	10^{274}	7,080	spectral normalize	**0.395**	**3.280**	160

Table 1: **Transformers with enforced Lipschitz constraints can match performance on NanoGPT.** The NanoGPT speedrun is a competitively tuned benchmark building on Karpathy's original replication of GPT-2 [Karpathy, 2022, Jordan et al., 2024a]. With the speedrun baseline as a starting point, we substitute Lipschitz transformer components and constrain weight norms to not exceed a given $\sigma_{\text{max}} \geq 0$. Unlike LipsFormer [Qi et al., 2023], our weight constraint methods enforce a Lipschitz constant chosen prior to training. To demonstrate, we train a 600-Lipschitz transformer to 21.2% accuracy. However, reaching accuracy on par with the baseline increases the Lipschitz bound to 10^{274}, computed as in Section 4.2. Our Lipschitz bounds may be loose, as suggested by the small maximum activation that we observe across a batch of 393K tokens. Per-run loss variance is 0.0008.

Table 1 summarizes our 145M parameter scale transformer results. In contrast to LipsFormer, our method guarantees a Lipschitz upper bound specified prior to training. An important lever is the maximum $\text{RMS} \to \text{RMS}$ norm $\sigma_{\text{max}} \geq 0$ we allow for the linear layers. Smaller σ_{max} correspond to smaller Lipschitz bounds but may lower performance, and vice versa. Two other variables that affect the Lipschitz constant are the attention logit scale and final logit scale. Using spectral normalization with $\sigma_{\text{max}} = 1$ and a final logit scale of 8 results in a 600-Lipschitz transformer that attains validation loss 5.047 and accuracy 21.2%. No activation in this model exceeds RMS norm 1, which could enhance stability during training. Setting $\sigma_{\text{max}} = 16$ enables reaching parity with the speedrun performance but increases the Lipschitz bound to 10^{276}. This setting trains for $4\times$ as many steps as the current speedrun record (but $2.4\times$ fewer than Karpathy's baseline).

5 Discussion

Despite high Lipschitz upper bounds, our transformers on NanoGPT exhibit low maximum activation entries (50-110) compared to the baseline (148K). Perhaps as a result, our models train stably without standard measures including layer norm, QK norm, or tanh logit softcapping. In future work, we are interested to test whether these low maximum activations hold potential for low-precision training and inference. We also wonder whether training would remain stable at larger scales.

For MLPs and small transformers, we find that using Muon improves the Lipschitz vs. performance tradeoff. Out of weight constraint methods we test, *spectral normalization*, *spectral soft cap*, and *spectral hard cap* compare favorably to standard weight decay. Perhaps surprisingly, on both CIFAR-10 and Shakespeare data, we achieve our best loss with Lipschitz-enforced models, potentially representing a training speed benefit.

Our work has several limitations. We did not find a principled way to select weight norm, final logit scale, and attention logit scale hyperparameters, instead relying on sweeps. Our Lipschitz bound also increases rapidly as depth increases, unless we constrain weights to unit norm. A different architecture, or insight beyond a global Lipschitz bound, could make progress on this problem.

In conclusion, this paper develops a method for training transformers with an enforced Lipschitz constant throughout training, extending earlier efforts focused on different architectures or only constraining at initialization. Lipschitz-certified transformers may be of particular interest for domains such as privacy, control, adversarial robustness, and low-precision training. Although training speed benefits fade in our NanoGPT speedrun experiments, we wonder whether at this scale Lipschitz-enforced training can be made faster than standard training.

Acknowledgements

We are grateful to Phillip Isola for the freedom to pursue this research. Thank you to Lambda Labs and Rami Seid for compute support.

References

Atish Agarwala, Samuel Stern Schoenholz, Jeffrey Pennington, and Yann Dauphin. Temperature check: Theory and practice for training models with softmax-cross-entropy losses. *Transactions on Machine Learning Research*, 2023. Cited on page 8.

Cem Anil, James Lucas, and Roger B. Grosse. Sorting out Lipschitz function approximation. In *International Conference on Machine Learning*, 2019. Cited on pages 1 and 3.

Martin Arjovsky, Amar Shah, and Yoshua Bengio. Unitary evolution recurrent neural networks. In *International Conference on Machine Learning*, 2016. Cited on page 1.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv:1607.06450*, 2016. Cited on page 2.

Peter Bartlett, Dylan J. Foster, and Matus Telgarsky. Spectrally-normalized margin bounds for neural networks. In *Neural Information Processing Systems*, 2017. Cited on pages 1 and 3.

Jeremy Bernstein. The Modula docs, 2025. URL `https://docs.modula.systems/`. MIT License. Cited on page 19.

Jeremy Bernstein and Laker Newhouse. Modular duality in deep learning. In *International Conference on Machine Learning*, 2025. Cited on pages 4 and 19.

Louis Béthune. *Deep Learning with Lipschitz Constraints*. PhD thesis, Université de Toulouse, 2024. Cited on pages 2 and 3.

Louis Béthune, Thibaut Boissin, Mathieu Serrurier, Franck Mamalet, Corentin Friedrich, and Alberto Gonzalez Sanz. Pay attention to your loss: Understanding misconceptions about Lipschitz neural networks. In *Neural Information Processing Systems*, 2022. Cited on pages 2 and 8.

Louis Béthune, Thomas Massena, Thibaut Boissin, Yannick Prudent, Corentin Friedrich, Franck Mamalet, Aurelien Bellet, Mathieu Serrurier, and David Vigouroux. DP-SGD without clipping: The Lipschitz neural network way. In *International Conference on Learning Representations*, 2024. Cited on pages 1 and 3.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018. URL `http://github.com/jax-ml/jax`. Cited on page 19.

Jonah Brown-Cohen, Geoffrey Irving, and Georgios Piliouras. Scalable AI safety via doubly-efficient debate. In *International Conference on Machine Learning*, 2024. Cited on page 3.

Franz Louis Cesista, YouJiacheng, and Keller Jordan. Squeezing 1–2% efficiency gains out of Muon by optimizing the Newton-Schulz coefficients, 2025. URL `http://leloykun.github.io/ponder/muon-opt-coeffs/`. Cited on page 20.

Moustapha Cisse, Piotr Bojanowski, Edouard Grave, Yann Dauphin, and Nicolas Usunier. Parseval networks: Improving robustness to adversarial examples. In *International Conference on Machine Learning*, 2017. Cited on pages 1, 4, 5, and 6.

Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, Rodolphe Jenatton, Lucas Beyer, Michael Tschannen, Anurag Arnab, Xiao Wang, Carlos Riquelme, Matthias Minderer, Joan Puigcerver, Utku Evci, Manoj Kumar, Sjoerd van Steenkiste, Gamaleldin F. Elsayed, Aravindh Mahendran, Fisher Yu, Avital Oliver, Fantine Huot, Jasmijn Bastings, Mark Patrick Collier, Alexey Gritsenko, Vighnesh Birodkar, Cristina Vasconcelos, Yi Tay, Thomas Mensink, Alexander

Kolesnikov, Filip Pavetić, Dustin Tran, Thomas Kipf, Mario Lučić, Xiaohua Zhai, Daniel Keysers, Jeremiah Harmsen, and Neil Houlsby. Scaling vision transformers to 22 billion parameters. In *International Conference on Machine Learning*, 2023. Cited on page 1.

Benoit Dherin, Michael Munn, Mihaela Rosca, and David GT Barrett. Why neural networks find simple solutions: The many regularizers of geometric complexity. In *Neural Information Processing Systems*, 2022. Cited on page 3.

Ahmed Taha Elthakeb, Prannoy Pilligundla, Fatemeh Mireshghallah, Alexander Cloninger, and Hadi Esmaeilzadeh. Divide and conquer: Leveraging intermediate feature representations for quantized training of neural networks. In *International Conference on Machine Learning*, 2020. Cited on page 3.

Mahyar Fazlyab, Alexander Robey, Hamed Hassani, Manfred Morari, and George J. Pappas. Efficient and accurate estimation of Lipschitz constants for deep neural networks. In *Neural Information Processing Systems*, 2019. Cited on pages 2 and 8.

Thomas Flynn. The duality structure gradient descent algorithm: Analysis and applications to neural networks. *arXiv:1708.00523*, 2017. Cited on page 3.

Florin Gogianu, Tudor Berariu, Mihaela Rosca, Claudia Clopath, Lucian Buşoniu, and Razvan Pascanu. Spectral normalisation for deep reinforcement learning: An optimisation perspective. In *International Conference on Machine Learning*, 2021. Cited on page 3.

Henry Gouk, Eibe Frank, Bernhard Pfahringer, and Michael J. Cree. Regularisation of neural networks by enforcing Lipschitz continuity. *Machine Learning*, 2021. Cited on pages 3 and 4.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Computer Vision and Pattern Recognition*, 2016. Cited on page 7.

Alex Henry, Prudhvi Raj Dachapally, Shubham Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Empirical Methods in Natural Language Processing*, 2020. Cited on page 2.

Yujia Huang, Huan Zhang, Yuanyuan Shi, J. Zico Kolter, and Anima Anandkumar. Training certifiably robust neural networks with efficient local Lipschitz bounds. In *Neural Information Processing Systems*, 2021. Cited on pages 5 and 6.

Su Jianlin. Thoughts from spectral norm gradient to new weight decay, Dec 2024. URL `https://kexue.fm/archives/10648`. Cited on pages 3 and 4.

Keller Jordan, Jeremy Bernstein, Brendan Rappazzo, @fernbear.bsky.social, Boza Vlado, You Jiacheng, Franz Cesista, Braden Koszarsky, and @Grad62304977. Modded-nanoGPT: Speedrunning the nanoGPT baseline, 2024a. URL `https://github.com/KellerJordan/modded-nanogpt`. MIT License. Cited on pages 2, 7, 8, 9, 19, 20, and 24.

Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024b. URL `https://kellerjordan.github.io/posts/muon/`. Cited on page 2.

Andrej Karpathy. nanoGPT. `https://github.com/karpathy/nanoGPT`, 2022. MIT License. Cited on pages 2, 8, 9, and 19.

Guy Katz, Clark Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In *International Conference on Computer Aided Verification*, 2017. Cited on page 3.

Hyunjik Kim, George Papamakarios, and Andriy Mnih. The Lipschitz constant of self-attention. In *International Conference on Machine Learning*, 2021. Cited on pages 2 and 7.

Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. Cited on pages 2 and 19.

Anders Krogh and John A. Hertz. A simple weight decay can improve generalization. In *Neural Information Processing Systems*, 1991. Cited on pages 2 and 3.

Tim Large, Yang Liu, Minyoung Huh, Hyojin Bahng, Phillip Isola, and Jeremy Bernstein. Scalable optimization in the modular norm. In *Neural Information Processing Systems*, 2024. Cited on pages 2, 3, 7, 8, 16, 19, and 23.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. Cited on page 2.

Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations*, 2018. Cited on pages 3 and 4.

Behnam Neyshabur, Srinadh Bhojanapalli, and Nathan Srebro. A PAC-bayesian approach to spectrally-normalized margin bounds for neural networks. In *International Conference on Learning Representations*, 2018. Cited on page 3.

Michael O'Connell, Guanya Shi, Xichen Shi, Kamyar Azizzadenesheli, Anima Anandkumar, Yisong Yue, and Soon-Jo Chung. Neural-fly enables rapid learning for agile flight in strong winds. *Science Robotics*, 2022. Cited on page 2.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Neural Information Processing Systems: Datasets and Benchmarks Track*, 2024. ODC-By 1.0 License. Cited on pages 7 and 19.

Thomas Pethick, Wanyun Xie, Kimon Antonakopoulos, Zhenyu Zhu, Antonio Silveti-Falls, and Volkan Cevher. Training deep learning models with norm-constrained LMOs. *arXiv:2502.07529*, 2025. Cited on pages 3 and 14.

Xianbiao Qi, Jianan Wang, Yihao Chen, Yukai Shi, and Lei Zhang. LipsFormer: Introducing Lipschitz continuity to vision transformers. In *International Conference on Learning Representations*, 2023. Cited on pages 1, 3, 8, 9, 18, and 23.

Mihaela Rosca, Theophane Weber, Arthur Gretton, and Shakir Mohamed. A case for new neural network smoothness constraints. In *NeurIPS Workshop on "I Can't Believe It's Not Better!"*, 2020. Cited on page 2.

Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *International Conference on Learning Representations*, 2014. Cited on pages 2 and 3.

Yusuke Tsuzuku, Issei Sato, and Masashi Sugiyama. Lipschitz-margin training: Scalable certification of perturbation invariance for deep neural networks. In *Neural Information Processing Systems*, 2018. Cited on pages 1 and 3.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Neural Information Processing Systems*, 2017. Cited on page 7.

Aladin Virmaux and Kevin Scaman. Lipschitz regularity of deep neural networks: Analysis and efficient estimation. In *Neural Information Processing Systems*, 2018. Cited on page 3.

Yuh-Shyang Wang, Tsui-Wei Weng, and Luca Daniel. Verification of neural network control policy under persistent adversarial perturbation. In *International Conference on Machine Learning*, 2020. Cited on page 2.

Lily Weng, Huan Zhang, Hongge Chen, Zhao Song, Cho-Jui Hsieh, Luca Daniel, Duane Boning, and Inderjit Dhillon. Towards fast computation of certified robustness for ReLU networks. In *International Conference on Machine Learning*, 2018. Cited on page 3.

Lily Weng, Pin-Yu Chen, Lam Nguyen, Mark Squillante, Akhilan Boopathy, Ivan Oseledets, and Luca Daniel. PROVEN: Verifying robustness of neural networks with a probabilistic approach. In *International Conference on Machine Learning*, 2019. Cited on page 3.

Tsui-Wei Weng, Pu Zhao, Sijia Liu, Pin-Yu Chen, Xue Lin, and Luca Daniel. Towards certificated model robustness against weight perturbations. In *AAAI Conference on Artificial Intelligence*, 2020. Cited on page 3.

Mitchell Wortsman, Peter J. Liu, Lechao Xiao, Katie Everett, Alex Alemi, Ben Adlam, John D. Co-Reyes, Izzeddin Gur, Abhishek Kumar, Roman Novak, Jeffrey Pennington, Jascha Sohl-Dickstein, Kelvin Xu, Jaehoon Lee, Justin Gilmer, and Simon Kornblith. Small-scale proxies for large-scale transformer training instabilities. In *International Conference on Learning Representations*, 2024. Cited on page 1.

Greg Yang, James B. Simon, and Jeremy Bernstein. A spectral condition for feature learning. *arXiv:2310.17813*, 2024. Cited on page 4.

Yuichi Yoshida and Takeru Miyato. Spectral norm regularization for improving the generalizability of deep learning. *arXiv:1705.10941*, 2017. Cited on pages 1, 2, 3, and 4.

Jiacheng You. Rapidly converging orthogonalizing Newton-Schulz iteration, 2025. URL `https://x.com/YouJiacheng/status/1893704552689303901`. Cited on pages 3 and 4.

Pengxiang Zhao, Ping Li, Yingjie Gu, Yi Zheng, Stephan Ludger Kölker, Zhefeng Wang, and Xiaoming Yuan. Adapprox: Adaptive approximation in Adam optimization via randomized low-rank matrices. *arXiv:2403.14958*, 2024. Cited on page 4.

A Coupling spectral cap to learning rate

This section proves Theorem 3.1: spectral soft cap bounds weight norm. We will derive a strength parameter that couples to the learning rate, because otherwise using odd polynomial approximation—rather than the ideal map $\min(\sigma_{\text{max}}, \sigma)$—accumulates errors when the learning rate falls below the approximation gap. We will prove the theorem using an equilbrium analysis: solving for the fixed point of a contractive map.

To warm up, there is a special case in which weight decay provably bounds the weight norm during training. It happens when the norm of the update is bounded by the learning rate, $\|\Delta W\| \leq \eta$. To see why, suppose weight decay is applied at every step of training and is linearly coupled to the learning rate as $\lambda\eta$ for some constant $\lambda > 0$. Subadditivity of norms guarantees $\|W + \Delta W\| \leq \|W\| + \|\Delta W\| \leq \|W\| + \eta$. The weights cannot increase further when the decay and the learning rate are in equilbrium: $\|W\| \cdot (1 - \lambda\eta) + \eta = \|W\|$. The equilibrium occurs at $\|W\| = 1/\lambda$. Therefore $1/\lambda$ is the maximum weight norm possible under standard weight decay, if the update norm is bounded above by η. Pethick et al. [2025] noted the same phenomenon.

To prove Theorem 3.1, we conduct a general equilibrium analysis to consider three effects: weight decay, optimizer step, and weight projection. The net effect of the three effects should decrease every singular value $\sigma \geq \sigma_{\text{max}}$. Let the weight decay λ be coupled to the learning rate η. Suppose the weight update is constrained to have norm $\|\Delta W\| \leq \eta$. Let $p(x)$ be an odd polynomial. Equilbrium occurs when

$$p(x \cdot (1 - \lambda\eta) + \eta) \leq x. \tag{3}$$

In words, apply weight decay, apply an optimizer step that will not increase singular values by more than η, and then apply the odd polynomial. If the singular value does not increase for all $x \in [0, \sigma_{\text{max}}]$, then the weight norm will never exceed σ_{max}.

Recall that $p_1(x) = x - \alpha x^3$ and $p_2(x) = x + \alpha x^3$.

When $p(x) = p_2(p_1(x))$, the equilibrium condition Equation (3) becomes

$$f(\alpha) = \left(k - \alpha k^3\right) + \alpha \left(k - \alpha k^3\right)^3 - \sigma_{\text{max}} \leq 0, \tag{4}$$

$$\text{where } \; k = \sigma_{\text{max}} \cdot (1 - \lambda\eta) + \eta. \tag{5}$$

We can consider only $x = \sigma_{\text{max}}$ because, in this case, larger x will decrease if a smaller x decreases. Here α is the free variable. Finding the smallest $\alpha > 0$ amounts to solving the quartic polynomial

$$-k^9\alpha^4 + 3k^7\alpha^3 - 3k^5\alpha^2 + k - \sigma_{\text{max}} = 0. \tag{6}$$

Any numerical solver can approximate α. This is the α that makes σ_{max} a fixed point under the overall training step. Connecting to the earlier equilibrium analysis, the special case $\alpha = 0$ is possible when already $k = \sigma_{\text{max}}$, or $\sigma_{\text{max}} = 1/\lambda$. While the coupling for spectral soft capping is not linear as in common implementations of AdamW, it succeeds at making tight weight norm bounds compatible with learning rate schedules that may tend to 0. Initializing the weights near σ_{max}, rather than strictly less than it, suffices for the bound to hold throughout training in practice.

One limitation of automatic coupling is that it may be stronger than necessary, because it assumes updates align perfectly with the weights in the worst case. If the learning rate is scheduled to 0, gradients may align less with the existing weights especially at the end, which can cause the weight norm to contract slightly.

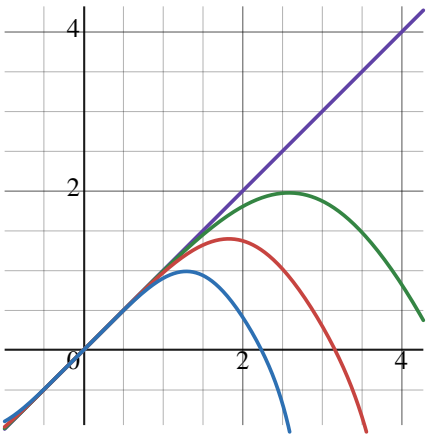

Figure 4: **Spectral soft cap** is a weight constraint method that applies the above odd polynomial to a weight matrix, which applies it to all singular values in parallel. First it applies $p_1(x) = x - \alpha x^3$, then it applies $p_2(x) = x + \alpha x^3$. The composition is depicted for $\alpha = 0.2$ (blue), $\alpha = 0.1$ (red), $\alpha = 0.05$ (green), $\alpha = 0$ (purple).

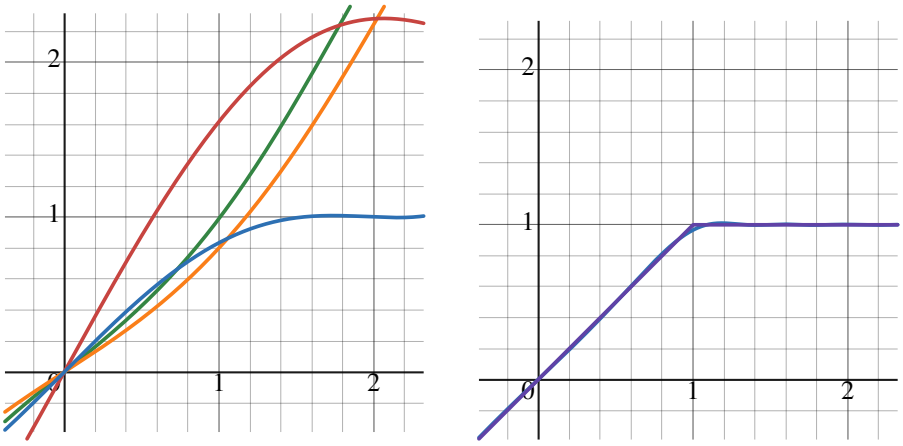

Figure 5: **Spectral hard cap** is a weight constraint method that applies the above odd polynomial to a weight matrix, which applies it to all singular values in parallel. Left: the four quintic polynomials. Right: the composition of the four quintic polynomials is designed to approximate $\min(1, x)$ on the range $x \in [0, 3]$.

B Proving an upper bound on the Lipschitz constant of a transformer

We elaborate on the algorithm sketched in Section 4.2 and prove a Lipschitz bound on attention. Our Lipschitz bounds are with respect to the max RMS norm over token positions, denoted $\|\cdot\|_{\infty\text{RMS}}$.

Recall the two primary ways Lipschitz constants L_f and L_g of two functions f and g interact:

- **Adding:** $f + g$ has Lipschitz constant at most $L_f + L_g$.
- **Composing:** $f \circ g$ has Lipschitz constant at most $L_f \cdot L_g$.

Step 1: Residual connections. Suppose that, before reaching a certain residual connection, a transformer maps input data x to $f(x)$ with Lipschitz constant L. Suppose the transformer has $2N$ residual connections. Let $\alpha = \frac{1}{2N}$. The residual connection acts on $f(x)$ as

$$[(1-\alpha)\cdot \mathsf{identity} + \alpha \cdot \mathsf{block}]\,(f(x)). \tag{7}$$

After the residual connection, the Lipschitz constant composes and adds to become at most

$$(1-\alpha)\cdot L + \alpha \cdot L \cdot L_{\mathsf{block}}. \tag{8}$$

Applying this formula sequentially upper bounds the Lipschitz constant of a transformer layer by layer. We now determine L_{block} for an MLP and attention block in terms of their weight norms.

Step 2: MLP. Our MLP composes $W_{\mathsf{out}} \circ (\mathsf{GeLU}/1.1289) \circ W_{\mathsf{in}}$. The Lipschitz constants of the two weight matrices are their norms $\|W_{\mathsf{out}}\|_{\text{RMS}\to\text{RMS}}$ and $\|W_{\mathsf{in}}\|_{\text{RMS}\to\text{RMS}}$, while $\mathsf{GeLU}/1.1289$ has Lipschitz constant 1 because we divide by the maximum derivative of GeLU. Overall, the Lipschitz bound for an MLP block is $L_{\mathsf{MLP}} \le \|W_{\mathsf{out}}\|_{\text{RMS}\to\text{RMS}}\|W_{\mathsf{in}}\|_{\text{RMS}\to\text{RMS}}/1.1289$.

Step 3: Attention. Let ℓ denote the token dimension. Let the queries, keys, and values be denoted by $(q,k,v) \in \mathbb{R}^{\ell\times d_Q} \times \mathbb{R}^{\ell\times d_Q} \times \mathbb{R}^{\ell\times d_V}$. Our attention block composes

$$\tfrac{1}{3}W_{\mathsf{out}} \circ F, \tag{9}$$

where function attention is denoted by $F = \text{softmax}(\frac{1}{d_Q}qk^\top + M)v$ for some mask M. As a consequence of the following theorem, if every attention input is unit norm, then functional attention is 1-Lipschitz. This property is what motivates scaling attention by $\frac{1}{d_Q}$ rather than $\frac{1}{\sqrt{d_Q}}$ inside the softmax. It also motivates scaling the attention output by $\frac{1}{3}$ to make it unit sensitivity. Functional attention is no longer 1-Lipschitz if its inputs are not unit norm. Recall that the shorthand notation $\|x\|_{\infty\text{RMS}}$ is the max RMS norm of a d-dimensional activation over l tokens, $x \in \mathbb{R}^{\ell\times d}$.

Theorem B.1 (Lipschitz bound on functional attention)**.** *Let $\diamond$ denote tensor contraction. Given any perturbations $\Delta q, \Delta k, \Delta v$ to the queries, keys, and values, functional attention satisfies*

$$\|\nabla F(q,k,v)\diamond(\Delta q,\Delta k,\Delta v)\| \le \|\Delta v\| + \|v\|(\|\Delta q\|\|k\| + \|q\|\|\Delta k\|), \tag{10}$$

where the norm is $\|\cdot\|_{\infty\text{RMS}} : \mathbb{R}^{\ell\times d} \to \mathbb{R}$, the max-over-tokens RMS *norm of the embedding vector.*

Proof. The argument mirrors the proof of Proposition 7 from the modular norm paper [Large et al., 2024]. We write the attention matrix as $A = \text{softmax}(\frac{1}{d_Q}qk^\top + M)$. Its derivative is $\Delta A = \nabla_{(q,k)}\text{softmax}(\frac{1}{d_Q}qk^\top + M)\diamond(\Delta q, \Delta k)$. The derivative of F splits into two terms,

$$\nabla F(q,k,v)\diamond(\Delta q,\Delta k,\Delta v) = A(\Delta v) + (\Delta A)v. \tag{11}$$

We call the maximum entry of A or ΔA its L^∞ operator norm, which comes into play by observing that $\|Ax\|_{\infty\text{RMS}} \le \|A\|_{\infty-\mathsf{op}}\|x\|_{\infty\text{RMS}}$. For the first term, note that $\|A\|_{\infty-\mathsf{op}} \le 1$ because softmax never exceeds 1. For the second term, Large et al. [2024] in Equation E.58 show that

$$\|\Delta A\|_{\infty-\mathsf{op}} \le \|\Delta q\|_{\infty\text{RMS}}\|k\|_{\infty\text{RMS}} + \|q\|_{\infty\text{RMS}}\|\Delta k\|_{\infty\text{RMS}}. \tag{12}$$

The result follows by applying these bounds to $\|A\|_{\infty-\mathsf{op}}\|\Delta v\|_{\infty\text{RMS}} + \|\Delta A\|_{\infty-\mathsf{op}}\|v\|_{\infty\text{RMS}}$. □

Step 4. Activation norm bounds. To apply the theorem, we now bound the input norm to attention. To do so we will track the maximum RMS norm of activations everywhere in the network. We do not use layer norm and therefore cannot reset activation norms to 1. Let $x_0, \ldots, x_{2N}$ denote all the activations, from the initial embedding x_0 through to the N alternating attention and MLP blocks acting via residual connections. Suppose the embedding layer maps tokens to have RMS norm at most 1, or $\|x_0\|_{\infty\text{RMS}} \le 1$. Attention and MLP increase the norm as follows:

- Attention computes $W_{\mathsf{out}} \circ (V, A)$ for some attention matrix A, where (V, A) is shorthand for functional attention. By definition V cannot increase the RMS norm of the embedding x_i at any token by more than its $\mathrm{RMS} \to \mathrm{RMS}$ operator norm, meaning $\|Vx_i\|_{\infty\mathrm{RMS}} \leq \|V\|_{\mathrm{RMS}\to\mathrm{RMS}}\|x_i\|_{\infty\mathrm{RMS}}$. The same bound applies to $(V, A)x_i$ by subadditivity of norms, since entries of the attention matrix A sum to 1 in the token dimension. Therefore attention can increase the activation norm by

$$\|(W_{\mathsf{out}} \circ (V, A))x_i\|_{\infty\mathrm{RMS}} \leq \|W_{\mathsf{out}}\|_{\mathrm{RMS}\to\mathrm{RMS}}\|V\|_{\mathrm{RMS}\to\mathrm{RMS}}\|x_i\|_{\infty\mathrm{RMS}}. \tag{13}$$

  In words, multiply the weight norms of W_{out} and V to get the maximum increase.

- The MLP computes $W_{\mathsf{out}} \circ (\mathsf{GeLU}/1.1289) \circ W_{\mathsf{in}}$. Therefore the MLP can increase activation norm by $\|W_{\mathsf{out}}\|_{\mathrm{RMS}\to\mathrm{RMS}}\|W_{\mathsf{in}}\|_{\mathrm{RMS}\to\mathrm{RMS}}/1.1289$, since $|\mathsf{GeLU}(x)| \leq |x|$ for all $x \in \mathbb{R}$.

- The residual connection acts like

$$\|(1-\alpha) \cdot x_i + \alpha \cdot \mathsf{block}(x_i)\|_{\infty\mathrm{RMS}} \leq (1-\alpha)\|x_i\|_{\infty\mathrm{RMS}} + \alpha\|\mathsf{block}(x_i)\|_{\infty\mathrm{RMS}}. \tag{14}$$

Algorithm to compute Lipschitz constant. Therefore, given the weight norms of all matrices in a transformer, we use the preceding results to compute its Lipschitz constant in two steps. First, we upper bound the activation norm everywhere in the network using Step 4. Second, we upper bound the Lipschitz constant using Steps 1-3. The Lipschitz bound after the final layer is what we refer to as the transformer's Lipschitz upper bound.

C Implementing LipsFormer and bounding its Lipschitz constant

To turn our enforced norm training into LipsFormer [Qi et al., 2023], we make the following changes:

1. Remove spectral soft cap and embed projections.
2. Use CenterNorm: mean subtraction with learnable entrywise scale and bias.
3. Use scaled-head cosine attention with $\epsilon = 10^{-6}$, $\tau = 12$, $\nu = 1$. Notably, the official implementation of LipsFormer uses $\epsilon = 0$. According to their Theorem 1, this choice may make a finite Lipschitz bound impossible. We set $\epsilon > 0$ to fix the issue.
4. Heuristically scale down attention output by $1/n_{\text{heads}}$ to match their implementation.
5. Insert residual connections with learnable strength α, initialized to $1/n_{\text{residual\_connections}}$.
6. Xavier normal initialize linear layers, then apply spectral normalization $W \mapsto W/\|W\|_*$.
7. Include drop path: every residual connection is skipped with $p = 0.5$ and, if taken, is scaled up by $1/(1-p)$, matching their official implementation which uses `nn.Dropout`.
8. Use weight decay 0.1, matching their implementation (not applied to scalar parameters).
9. Use the Muon optimizer to give LipsFormer the fairest comparison, copying hyperparameters from our run. We tested training with AdamW for all parameters, an exact replication, but found performance degraded sigificantly: after 1770 steps, validation loss was 4.86 (compared to 3.61) and validation accuracy was 0.227 (compared to 0.301).
10. For non-weight-matrix parameters, use Adam hyperparameters $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ to match their implementation.
11. Use cosine learning rate schedule with decay to 0 to match their implementation.

Bounding the Lipschitz constant of LipsFormer. In Table 1, we report that our trained implementation of LipsFormer has a Lipschitz upper bound of 10^{130}. To calculate this value, we use the final weight norms of the MLP and attention blocks to bound the Lipschitz constant of each residual block, relying on LipsFormer's Theorem 1:

$$\mathsf{Lip}(\text{SCSA})_2 \leq 2N(N-1)\nu\tau\epsilon^{-\frac{1}{2}}\|W^K\|_2 + 2(N-1)\nu\tau\epsilon^{-\frac{1}{2}}\|W^Q\|_2 + 2N\nu\epsilon^{-\frac{1}{2}}\|W^V\|_2.$$

Using $N = 128$ (head dimension), $\tau = 12$, $\nu = 1$, and empirical weight norms, we calculate the Lipschitz constant for every layer. We use the maximum entry of the learned residual strength α, which is an entrywise multiplication, to convert the layerwise bounds into a final bound

$$\mathsf{Lip}(F) \leq \prod_{s=1}^{S}\prod_{m=1}^{S}(1 + \alpha_{s,m}\mathsf{Lip}(f_{s,m})),$$

which we take from their Equation 19. Alpha has typical maximum entries around 0.5 for attention connections and 0.15 for MLP connections. With $\epsilon = 10^{-6}$, we compute a final Lipschitz bound of 1.97×10^{129}.

D Experimental details

This section gives experimental details for all results in the paper. The three categories of experiments we run are MLP training, Shakespeare transformer training, and NanoGPT speedrun training.

Datasets.

- For MLP training we use the CIFAR-10 dataset Krizhevsky [2009] with the standard train and test splits and no data augmentation. We do not shuffle the order of batches.
- For Shakespeare transformer training we use Karpathy's 1M character-level dataset with standard training and validation splits [Karpathy, 2022]. We shuffle the order of batches.
- For NanoGPT speedrun transformer training we use the FineWeb10B dataset [Penedo et al., 2024] loaded in the standard order. We use the same validation split as the modded NanoGPT speedrun benchmark [Jordan et al., 2024a].

Compute requirement. All our experiments can run on a V100, A100, or H100 GPU in less than 5 minutes, except the NanoGPT speedrun transformer which requires 8xH100 and runs in 5-10 minutes.

Modula library. For MLP and Shakespeare experiments, we use JAX [Bradbury et al., 2018] on top of the Modula library [Large et al., 2024, Bernstein, 2025]. We implement our own model components. Our AdamW implementation does not include bias correction, although the discrepancy decays rapidly after aronud 20 steps because we use $\beta_1 = 0.9, \beta_2 = 0.95$ in all experiments except one, not reported, in which we determine that this is a good setting for the momentum EMAs.

MLP experiments. All MLPs we train are width 256 and depth 3 (i.e., one hidden layer) with ReLU activations and no bias on data from CIFAR-10. We use batch size 512 and a linear learning rate schedule that decays to 0 in all experiments. Modula's mass calculation causes the effective learning rate to be scaled by $1/3$. We train for 50 epochs except in one case, when we train for 20 epochs for the models in Figure 2. We zero-initialize the final layer. We train all models in float32 precision and run the weight constrain methods in float32 precision. We experimented with lower precision and found comparable metrics across the board for bfloat16 training. We set seed 0 and store all hyperparameters and log information to enhance reproducibility.

Shakespeare experiments. All transformers we train for Shakespeare are width 256 with 3 blocks (attention + MLP), no bias, and four attention heads. The out projection in each attention and MLP block is initialized to zero. We use sequence length 256 and batch size 64 to match the baseline from [Karpathy, 2022], except we train for 2000 steps while Karpathy trains for 5000 steps. We set Modula's blocks mass parameter to 32 to cause 95% of the feature learning to occur in the transformer blocks. We determined this ratio by sweeping the blocks mass, which controls the ratio of learning rate between the two embedding layers and the transformer blocks. Training with Muon means applying Muon to all linear layer weight matrices (including the final logit head) but normalizing the columns of embedding gradient, as suggested by the $\ell_1 \to \mathrm{RMS}$ duality map [Bernstein and Newhouse, 2025]. We were concerned that rare tokens may cause the momentum buffer to dualize columns to full strength updates for hundreds of steps until the column decays to exactly zero, so we tested whether capping the maximum inflation factor for the embedding column normalization could help. We tested maximum factors in the set $\{1, 4, 16, \ldots, 65536\}$ across 8 seeds and found no significant difference. We choose to maximally multiply each column by 16 during the dualization step. Finally, we found that to train to the validation losses reported we had to use a trick: we decayed the learning rate by a factor of 1/2 per residual layer, causing later layers to train more than earlier layers. This change is implemented by setting the sensitivity of the Mul module in Modula to 1. We do not know why this trick is necessary.

Figure 1 sweeps over the following hyperparameters:

- MLPs on CIFAR-10: we test the following combinations of optimzer and weight constraint method: (AdamW, weight decay), (AdamW, spectral weight decay), (AdamW, spectral normalization), (AdamW, Stiefel manifold projection), (AdamW, spectral hammer), (Muon, weight decay), (Muon, spectral weight decay), (Muon, spectral normalization), (Muon, stiefel manifold projection), (Muon, spectral hammer), (Muon, spectral soft cap), (Muon, spectral hard cap). For AdamW, we vary the weight decay and spectral weight decay parameters with 10 points in log-space from 10^{-2} to 10^{0}. For Muon we vary the weight decay parameter with 10 points in log-space from 10^{-3} to 10^{0} and the spectral weight

decay parameter with 10 points in log-space from 10^{-2} to 10^{0}. For AdamW with spectral normalization, Stiefel manifold projection, and spectral hammer, we vary the maximum weight norm in the set $\sigma_{\text{max}} \in \{2, 3, 4, 5, 6, 7, 8\}$. For Muon with spectral normalization, Stiefel manifold projection, spectral soft cap, and spectral hard cap, we vary the maximum weight norm in the set $\sigma_{\text{max}} \in \{4, 5, 6, 7, 8, 9, 10\}$. For Muon with spectral hammer we vary the maximum weight norm in the set $\sigma_{\text{max}} \in \{1, 2, 3, \ldots, 10\}$. For AdamW with all methods we sweep 16 learning rates in log-space between 10^{-5} and $10^{-0.5}$. For Muon we sweep 16 learning rates in log-space between 10^{-2} and 10^{1} for all methods except for spectral hammer, where we use these learning rates for $\sigma_{\text{max}} \in \{4, 5, 6, 7, 8, 9, 10\}$, and use 16 learning rates in log-space between 10^{-3} and 10^{0} for $\sigma_{\text{max}} \in \{1, 2, 3\}$. Overall, this results in 1,610 total combinations, 682 with AdamW and 928 with Muon.

- Transformers on Shakespeare: we test the following combinations of optimizer and weight constraint method: (AdamW, weight decay), (AdamW, spectral normalize), (AdamW, spectral hammer), (Muon, weight decay), (Muon, spectral normalize), (Muon, spectral soft cap). For spectral normalize, spectral hammer, and spectral soft cap, we vary the maximum weight norm in the set $\sigma_{\text{max}} \in \{1.0, 1.2, \ldots, 2.8, 3.0\}$. For the baseline, we vary weight decay in the set $\lambda \in \{2/3, 0.5, 0.4, 0.3, 0.2, 0.1, 0.05, 0.03, 0.01, 0\}$. For AdamW we sweep 16 learning rates between $10^{-4.5}$ and $10^{-1.5}$. For Muon, we sweep 12 learning rates between $10^{-1.5}$ and $10^{1.5}$. We ran tests before to find ranges that cover the optimal learning rate.

Figure 2 reports adversarial examples and dataset-wide statistics from two models trained for 20 epochs. The AdamW model is trained with learning rate 8.1×10^{-3} and weight decay $\lambda = 0.1$. The Muon model is trained with learning rate 2.3×10^{-1} and weight decay $\lambda = 0$, using the spectral soft cap method with a weight constraint of $\sigma_{\text{max}} = 3$.

The left panel of Figure 3 visualizes the same data from the experiment for Figure 1, but focuses only on MLPs trained with Muon on CIFAR-10. The middle and right panels use the Muon optimizer, with the following tuples of (weight constraint method, maximum singular value, weight decay, spectral weight decay, learning rate): (weight decay, N/A, 0.1, 0, 1.585), (spectral weight decay, N/A, 0, 0.05, 0.157), (spectral normalization, 6, 0, 0, 1.0), (Stiefel manifold projection, 5, 0, 0, 1.0), (spectral hammer, 4, 0, 0, 0.398), (spectral soft cap, 6, 0, 0, 0.398), (spectral hard cap, 5, 0, 0, 0.631).

NanoGPT experiments. Following the Modded-NanoGPT speedrun standard [Jordan et al., 2024a], our training runs print log files with the full source code required to reproduce the results. We briefly summarize the changes we made to convert the NanoGPT speedrun record (as of May 2025) into our method:

- Every step, RMS normalize the embedding columns.
- Initialize all linear layer weight matrices to be orthogonal.
- Reparameterize residual connections according to Equation (1): $\frac{L-1}{L}x + \frac{1}{L}\mathsf{block}(x)$ residual connections, where $L = 24$ is the number of residual connections.
- Reparameterize attention according to Equation (2): $\frac{1}{3}$ overall scale on the attention output and $1/d_{\text{head}}$ scale inside the softmax.
- Every step, apply spectral soft cap (or spectral normalize) to every linear layer weight matrix based on a prespecified maximum desired weight norm σ_{max}.
- Use different orthogonalization coefficients that at most inflate a singular value to 1.14502. Therefore, the maximum update norm we pass to the strength parameter solver for learning rate coupling in spectral soft cap is $\eta \cdot 1.14502 \cdot 1.05$ with an extra factor of 1.05 to be safe around numerical precision errors. The iteration is derived by modifying the method in [Cesista et al., 2025].
- Remove U-net structure.
- Use $\mathsf{GeLU}/1.1289$ instead of ReLU^2.
- Switch the dimension scaling in Muon to be $\sqrt{\mathsf{fan\_out}/\mathsf{fan\_in}}$ instead of $\max(1, \sqrt{\mathsf{fan\_out}/\mathsf{fan\_in}})$.
- Remove RMS normalization: the model is now Lipschitz continuous.

- Add back the 7th attention layer (which was removed in the speedrun).
- Weight projections are run in bfloat16 (which we found to slightly improve performance). Spectral normalization uses 2 iterations, meaning that weight norms can exceed the specified maximum $\sigma_{\max}$ due to approximation error; in practice weights with norms enforced by spectral normalization exceed the specified maximum by around 10%.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Section 3 reports our comparisons between the weight constraint methods, as summarized in the contributions statement in the introduction, and Section 4 reports the results about transformers trained with an enforced Lipschitz constant, as stated in the abstract. We caveat that we do not attain competitive performance without an astronomical Lipschitz bound.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: We caveat that our methods only reach competitive performance with the unconstrained baseline with astronomical Lipschitz constants. We discuss other limitations in Section 5 may not make the best use of a given Lipschitz budget,

   Guidelines:

   - The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: The two theorems we state are each proved in the appendices: Appendix A for Theorem 3.1 and Appendix B for Theorem B.1. We reference related proofs from LipsFormer [Qi et al., 2023] and modular norm theory [Large et al., 2024].

   Guidelines:

   - The answer NA means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Appendix D lists all experimental details. Appendix B describes our algorithm to compute the Lipschitz constant of our transformer.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No]

   Justification: We will release full code upon submission of the paper (i.e., MLPs and Shakespeare transformer). However, here is an anonymized PyTorch script that reproduces our Lipschitz-enforced NanoGPT speedrun with weight norm constrained to 8 using our spectral soft cap method: txt file log + full source code. Environment setup instructions are available in [Jordan et al., 2024a].

   Guidelines:

   - The answer NA means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: Appendix D reports all experimental details and is referred to in the main body of the paper.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

Justification: We report error intervals in Figure 2 and in the NanoGPT speedrun experiments where the validation loss variance is 0.0008. For our other main claims, including the method comparison, we look at trendlines over hundreds of training runs but do not analyze the statistical significance.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix D describes our compute resources. All our experiments can be replicated on an H100 (training runs take <5 minutes), except the NanoGPT speedrun which requires an 8xH100 (training runs take 5-10 minutes).

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: We read all points in the ethics checklist and declare that we conform to them.

   Guidelines:

   - The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Our paper is unlikely to have a strong societal impact; however, the broader goal of Lipschitz constrained neural networks is to enable positive applications such as safe actuator control and differential privacy, as discussed in Section 2.

    Guidelines:

    - The answer NA means that there is no societal impact of the work performed.
    - If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: [NA]

    Justification: We do not release any applicable data or models.

    Guidelines:

    - The answer NA means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We cite all assets we use and mention the license in each citation.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [NA]

Justification: We do not introduce new assets.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: Our paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: Our paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [NA]

Justification: Our ideas, results, and writing are due to humans, not LLMs.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (`https://neurips.cc/Conferences/2025/LLM`) for what should or should not be described.